

ESMC Model and Front-End Engineering Report

National Taiwan Ocean University · 6 October 2026

Summary

This report covers two strands of work on the ADAM portal: the ESMC protein-language-model classifier, and the front-end engineering of the web application. Both are complete and deployed.

The headline research result is negative, and is deliberately reported as such: fine-tuning ESMC was tested against the existing frozen-embedding approach under controlled conditions and performed measurably worse. The frozen approach was retained on that evidence. All accuracy figures below come from stratified 5-fold cross-validation, not from a single favourable train/test split.

Part I — The ESMC Classifier

1.1 Model and configuration

ESMC (ESM Cambrian) is EvolutionaryScale's newest protein language model. It is loaded through a community port that is directly usable with the standard transformers library, avoiding a dependency on the upstream native SDK.

Property	Value
Model	Synthyra/ESMplusplus_small (ESM Cambrian)
Parameters	332,997,184
Transformer blocks	30
Embedding dimension	960
Revision	Pinned to commit 3399aae2...
Mode	Frozen — weights are never updated

The revision is pinned deliberately. Without pinning, the loader resolves to the latest upstream commit, which would silently substitute different weights beneath an already-trained classifier head and invalidate the reported accuracy.

1.2 Method

Classification runs in two stages rather than as one end-to-end network:

- Each sequence is passed once through the frozen ESMC model and mean-pooled using the attention mask, converting a variable-length peptide into a fixed 960-dimensional vector.
- That vector is classified by a lightweight head: a StandardScaler feeding a support vector machine with an RBF kernel, probability output enabled, and balanced class weighting.

Because the language model is frozen, every sequence need only be embedded once. The embeddings are cached to disk and reused across all five cross-validation folds and all subsequent experiments.

1.3 Measured accuracy

Cross-validated accuracy: $94.84\% \pm 0.48\%$, from stratified 5-fold cross-validation over 12,000 samples. Each fold trains on 9,600 sequences and is evaluated on the 2,400 it has never seen.

Fold	1	2	3	4	5
Accuracy	93.92%	94.96%	94.92%	95.33%	95.08%

Per-class performance on a representative fold, which confirms the model is not simply exploiting the 58/42 class imbalance:

Class	Precision	Recall	F1
Non-AMP	0.93	0.92	0.93
AMP	0.94	0.95	0.95

1.4 Research finding: fine-tuning was tested and rejected

A reasonable objection to the frozen approach is that fine-tuning the language model should perform better. This was tested directly rather than assumed. Both arms used identical data and identical train/test splits, so the comparison isolates the single variable:

Approach	Accuracy	Verdict
Frozen embeddings + SVM head	95.08%	Retained
Fine-tuning last 2 transformer blocks	93.50%	Rejected

Fine-tuning was 1.58 percentage points worse. The cause is model capacity, not faulty implementation. Unfreezing two blocks exposes 22,129,920 trainable parameters to only 4,800 training examples — roughly 4,600 parameters per example. The model memorises instead of generalising, and this is visible in the training curve: training loss fell steadily while held-out accuracy declined.

Full fine-tuning of all 30 blocks was ruled out separately on cost. Benchmarked at 16.183 seconds per sequence on this hardware, a 5-fold cross-validation would require approximately 539 hours.

Conclusion: for a dataset of this size on CPU-only hardware, frozen embeddings with a small trained head is both the more accurate and the more practical choice.

1.5 Computational cost

Stage	Cost
Embedding all sequences (one-off)	8,351.8 s (~2.3 hours), ~0.77 s per sequence
Re-embedding for later experiments	0 s — served from cache
Training the classifier head	~37 s per fold

Caching the embeddings is what made the fine-tuning comparison affordable: the second experiment reused the first experiment's embeddings instead of recomputing 2.3 hours of work.

Part II — Front-End Engineering

2.1 Architecture

The portal is a React application built with Vite and routed with React Router, served by an Express backend, which in turn calls a persistent Flask service that holds the machine-learning models in memory. Keeping the models resident in a long-running Python process avoids paying model load time on every prediction request.

2.2 Route-level code splitting

Only the landing page is bundled eagerly. All twelve remaining routes — search, clustering, the four model pages, the ensemble, the guide, structure search and sequence search — are split into separate chunks loaded on demand, so a first-time visitor does not download the entire application before seeing anything.

The loading placeholder is painted in the page background colour, so a chunk load reads as a brief pause rather than a white flash between the header and footer.

2.3 Component consolidation

The four model prediction pages were originally near-identical copies of roughly 308 lines each. They now share a single parameterised component and declare only their differences:

Measure	Before	After
Per model page	~308 lines	28 lines
Shared component	none	1 (552 lines)
Files to edit to change prediction UI	4	1

2.4 Progress reporting

Multi-sequence predictions can take a long time, so the interface reports genuine progress rather than displaying an indeterminate spinner. Sequences are submitted one at a time specifically so that each completed reply is real evidence of work done:

- A progress bar shows completed sequences against the total.
- The estimated time remaining is extrapolated from actual measured elapsed time, not from a fixed guess.
- A Stop button cancels an in-flight run.
- On multi-sequence runs the view anchors to the progress bar so the user can watch it advance.
- The animation is disabled for users who have requested reduced motion.

2.5 Result page rebuild

The peptide result page was rebuilt around the sequence itself. Residues are shown in a monospace face grouped in tens with net charge displayed, there is a one-click Copy FASTA action, and a handoff that carries the sequence directly into any prediction model. Database cross-references to PDB, UniProt and PubMed are now live links, and empty fields are collapsed behind a toggle instead of occupying space. Page height fell from 2,536 px to 1,514 px.

2.6 Defects identified and corrected

Several defects were found during this work that were actively costing the user information:

Defect	Effect on the user
A CSS colour variable used but never defined	Invisible search icon; white text on a white badge
Hard-coded database size in the page text	Published an incorrect figure; now read live from the server
CSS rule declared after the media query overriding it	Mobile navigation menu was unreachable
Grid track sizing using 1fr instead of minmax(0, 1fr)	Result metadata overflowed its container
Unescaped commas in exported data	Corrupted CSV downloads
Scroll-reveal animation with no fallback	Content stayed permanently invisible if the observer never fired
Inconsistent import filename casing	Would have broken the build on Linux

2.7 Ensemble integration

ESMC contributes to the combined prediction with a weight of 0.94, second only to the fine-tuned FLM model. Each model casts an AMP / non-AMP vote weighted by its own held-out accuracy, so stronger models count for more. The interface states these weights openly and warns that scores near 50% are genuinely uncertain rather than confident — precisely the region where the models disagree most.

Status note

The ESMC figures in this report are final and deployed. The FLM model is currently being retrained, and its accuracy figure should not be published until that run completes.